

CSC 2114 Artificial Intelligence

Practical Session /Assignment 1: Makerere Campus Route Finder (DFS and BFS)

Duration: 2 hours | Language: Python 3 (standard library only) | Submission: MUELE
Deliverables: (1) `route_finder.py` (2) `report.pdf`

1. The Problem

A visitor is standing at one school on the Makerere University main campus and wants to walk to another. Your agent must find a route through the campus's named schools and landmark buildings.

The user supplies a **source school**, a **destination school** and a **search algorithm** (`BFS` or `DFS`). Your program prints the route it finds.

Cost model. We deliberately ignore physical distance. **Every walk between two directly connected places costs exactly 1**, so the cost of a route is its number of hops. The optimal route is the one with the fewest hops.

2. The Campus Graph (given)

The graph has 14 schools and 4 landmark buildings. Landmarks can be walked *through* but are not schools. Connections are two-way. The layout is a simplified teaching model built for this exercise, so **do not change it**, and do not treat it as a survey-accurate map.

Type	Places
Landmarks	Main Building, Freedom Square, Main Library, Central Teaching Facility
Schools	CoCIS; School of Engineering; School of the Built Environment; Margaret Trowell School of Industrial and Fine Art; School of Law; School of Education; School of Social Sciences; School of Liberal and Performing Arts; School of Economics; School of Statistics and Planning; College of Natural Sciences; School of Food Technology, Nutrition and Bioengineering; School of Agricultural Sciences; School of Veterinary Medicine

Connections (24 edges):

```

EDGES = [
    ("Main Building", "Freedom Square"), ("Main Building", "Main Library"),
    ("Main Building", "School of Social Sciences"),
    ("Main Building", "School of Liberal and Performing Arts"),
    ("Freedom Square", "Main Library"), ("Freedom Square", "School of Law"),
    ("Freedom Square", "Central Teaching Facility"),
    ("Main Library", "School of Education"), ("Main Library", "School of Economics"),
    ("Central Teaching Facility", "CoCIS"),
    ("Central Teaching Facility", "School of Statistics and Planning"),
    ("CoCIS", "School of Statistics and Planning"), ("CoCIS", "School of Engineering"),
    ("School of Engineering", "School of the Built Environment"),
    ("School of Engineering", "Margaret Trowell School of Industrial and Fine Art"),
    ("School of the Built Environment", "Margaret Trowell School of Industrial and Fine
Art"),
    ("Margaret Trowell School of Industrial and Fine Art", "School of Liberal and
Performing Arts"),
    ("School of Education", "School of Social Sciences"),
    ("School of Education", "College of Natural Sciences"),
    ("College of Natural Sciences", "School of Food Technology, Nutrition and
Bioengineering"),
    ("School of Food Technology, Nutrition and Bioengineering", "School of Agricultural
Sciences"),
    ("School of Agricultural Sciences", "School of Veterinary Medicine"),
    ("School of Statistics and Planning", "School of Economics"),
    ("School of Economics", "School of Law"),
]

```

The graph contains several cycles (for example Main Building, Freedom Square and Main Library form a triangle). That is deliberate: it is what makes repeated states a real problem.

3. Practical Session Plan

Part A: Formulate (30 min)

In your report, write the five-component definition of this problem. Be precise, and state the *type* of each component (set, function, Boolean test):

1. **Initial state**
2. **Actions(s)**, including how the set of legal actions is read off the graph
3. **Transition model** `Result(s, a)`
4. **Goal test**
5. **Path cost**, and why the step cost is 1

Then answer in two or three sentences each:

- **Abstraction.** List three real-world details of walking across campus that your state deliberately ignores (for example distance, slope, rain, traffic). Justify why dropping them is *valid* and *useful*.
- **State versus node.** Give one example from this graph of a single state that appears in two different nodes of the search tree.

Part B: Build and Search (75 min)

Implement `route_finder.py` following the lecture's design. **Do not use** `networkx` or any search library.

B1. Data structures (20 min)

- A `Node` class with the four fields from the lecture: `STATE`, `PARENT`, `ACTION`, `PATH-COST`.
- A function `solution(node)` that follows `PARENT` pointers back to the root and returns the list of places in order.
- A `reached` table (a dictionary mapping state to node) used for graph search.

B2. Generic search loop (30 min)

Implement **one** search function that follows the lecture's seven steps and takes the strategy as a parameter. BFS and DFS must differ *only* in the frontier discipline:

Algorithm	Frontier	Remove from
BFS	queue (FIFO)	front
DFS	stack (LIFO)	back

Requirements:

- Apply the **goal test** when a node is removed from the frontier (step 4 of the lecture algorithm).
- **Nodes expanded** means nodes removed from the frontier that are *not* the goal and have their children generated.
- **Deterministic ordering.** So that results can be marked, always consider a place's neighbours in **alphabetical order**. For DFS, this means the alphabetically first neighbour must be the *first one explored*. Think about what that implies for the order in which you push children onto a stack.
- Provide a switch (a function argument or command-line flag) to turn the `reached` table **off**, giving tree-like search. Add a safety cap of 10,000 expansions so that this mode terminates.

B3. User interface (15 min)

The program must let the user choose the source, destination and algorithm through interactive prompts:

It must **print** the solution in this format:

```
Algorithm: BFS
Route: <place 1> -> <place 2> -> ... -> <place n>
Cost (hops): <integer>
Nodes expanded: <integer>
```

Input handling (marked):

- Place names are matched case-insensitively and ignoring leading or trailing spaces.
- An unknown place or algorithm prints a clear message and lists the valid places. It must not crash.
- If source equals destination, print a route of one place with cost 0.
- If no route exists, print `No route found.`

B4. Verify the reached table (10 min)

Run DFS between two schools in **both** modes: with `reached` and without it. Record what happens in the tree-like run (it should hit your 10,000 cap). Explain in your report *why* the cycles cause this and how `reached` prevents it.

Part C: Submit (15 min)

Run **both** algorithms on **ten** distinct source and destination pairs of your choice, and report the results in a table:

#	Source	Destination	Algorithm	Route	Cost (hops)	Nodes expanded
---	--------	-------------	-----------	-------	-------------	----------------

That is twenty rows. Your ten pairs must include:

- at least two pairs where **BFS and DFS return different routes**,
- at least one pair of schools that are **directly connected**,
- at least one pair at least **six hops** apart.

4. Report (max 4 pages, PDF)

1. **Problem formulation** (Part A answers)

2. **Design**: a short description of your `Node`, `reached` and frontier, plus a diagram or pseudocode of the generic search loop

3. Results table (Part C)

4. Analysis (the substantive part):

- Using your results, explain why BFS always returns a fewest-hops route on this graph. State the condition from the lecture that makes this true.
- Find a pair where DFS returns a **longer** route than BFS. Draw both routes on the graph and explain *why* DFS stopped at the longer one.
- Compare nodes expanded by the two algorithms. Is DFS always cheaper? Give a pair where it is not.
- Explain the memory trade-off: what in this graph would make BFS's frontier grow, and why does the lecture say memory, not time, is what limits BFS in practice?
- Discuss **one limitation** of the equal-cost assumption. What would change if edges carried real walking distances? (Name the algorithm from the lecture's best-first table that you would use. You do not need to implement it.)

5. **Reflection:** what one thing would you change about the campus model to make it more realistic?

5. Marking Scheme (100 marks)

Component	Marks
Part A: five-component formulation, abstraction and state versus node	15
<code>Node</code> class, <code>solution()</code> via PARENT pointers, <code>reached</code> table	15
Single generic search loop; BFS and DFS differ only in frontier	20
Correct routes, costs and expansion counts (alphabetical ordering)	15
Interface, output format and input error handling	10
Tree-like versus graph search demonstration (B4)	5
Results table meets all Part C conditions	5
Analysis quality and use of evidence	12
Code clarity, comments, and the report's presentation	3

Automatic deductions: hard-coded routes (0 for correctness marks); use of a search or graph library (−20); unresolved plagiarism referral.

6. Notes

- Distances, coordinates and weights **must not** influence your algorithms. Every hop costs 1.
- Cost is measured in hops, so DFS's route is never cheaper than BFS's, but it can be equal.
- Submit only `route_finder.py` and `report.pdf`, named `<StudentNo>_route_finder.py` and `<StudentNo>_report.pdf`.

Before next week: read Russell & Norvig 4th ed., sections 3.4 and 3.5. Think about this: if walking distances were added to the edges, on which pair from your results table would BFS stop returning the cheapest route?